

Video Summary

This Telusko lesson introduces Python lists as flexible, mutable data structures for storing multiple items in one variable. It covers creating lists, mixing data types, nested lists, indexing, slicing, list methods, ordering, and numeric helper functions.

1

Creating Lists

Lists are created with square brackets. A list can hold numbers, strings, mixed data types, and even another list.



Square brackets

Use [] to collect values in a single variable.



Mixed values

A list can store integers, strings, floats, and more.



Nested lists

A list can contain another list as one of its elements.

List creation examples

```
>>> nums = [25, 12, 36, 95, 14]
>>> names = ["navin", "kiran", "harsh"]
>>> values = [33, "Telusko", 5.5]
>>> pair = [nums, names]
>>> pair[1][0]
'navin'
```

2

Indexing

Each list element has a position. Positive indexes count from the beginning, and negative indexes count backward from the end.

Indexing examples

```
>>> nums = [25, 12, 36, 95, 14]
>>> nums[0]
25
>>> nums[2]
36
>>> nums[-1]
14
```

3

Slicing

Slicing creates a new list from part of an existing list. In [start:end], the start index is included and the end index is excluded.

Slice	Meaning	Result for nums
nums[1:4]	Start at index 1 and stop before index 4.	[12, 36, 95]
nums[:3]	Start at the beginning and stop before index 3.	[25, 12, 36]
nums[2:]	Start at index 2 and continue to the end.	[36, 95, 14]
nums[-3:]	Take the last three elements.	[36, 95, 14]

List slicing rule

Slicing does not change the original list. It creates a new list containing the selected elements.

4

List Mutability

Lists are mutable, so their contents can be changed after creation. This is different from strings, which cannot be changed in place.

Mutability examples

```
>>> nums = [25, 12, 36, 95, 14]
>>> nums[2] = 77
>>> nums
[25, 12, 77, 95, 14]
```

5

Adding Items

Python gives lists several methods for adding values. Use `append()` for the end, `insert()` for a position, and `extend()` to add many items.

Method	What it does	Example
<code>append(x)</code>	Adds one item at the end.	<code>nums.append(45)</code>
<code>insert(i, x)</code>	Adds one item at index <code>i</code> .	<code>nums.insert(2, 77)</code>
<code>extend(seq)</code>	Adds each item from another sequence.	<code>nums.extend([29, 88])</code>

Adding rule

`append()` and `insert()` add one object. `extend()` adds each element from another sequence.

6

Removing Items

Lists can remove values by item, by position, or with the `del` keyword. Choose the operation based on what you know.

Operation	What it does	Example
<code>remove(x)</code>	Removes the first matching value.	<code>nums.remove(12)</code>
<code>pop(i)</code>	Removes and returns an item by index.	<code>nums.pop(2)</code>
<code>del</code>	Deletes an item or slice by position.	<code>del nums[1]</code>

Removing examples

```
>>> nums = [25, 12, 36, 95, 14]
>>> nums.remove(12)
>>> nums.pop(2)
95
>>> del nums[1]
>>> nums
[25, 14]
```

Removing rule

`remove()` searches by value. `pop()` and `del` work by index; `pop()` returns the removed item.

7

Organization and Functions

`reverse()` and `sort()` change the order of a list. Numeric lists also work with useful built-in functions like `min()`, `max()`, and `sum()`.

rev**reverse()**

Reverses the list order in place.

sort**sort()**

Sorts values into ascending order in place.

123**min max sum**

Calculate smallest, largest, and total values.

Ordering and functions

```
>>> nums = [25, 12, 36, 95, 14]
>>> min(nums), max(nums), sum(nums)
(12, 95, 182)
>>> nums.reverse()
>>> nums
[14, 95, 36, 12, 25]
>>> nums.sort()
>>> nums
[12, 14, 25, 36, 95]
```

In-place changes

`reverse()` and `sort()` update the existing list. Use another approach when you need a separate copy.

8

Code Solution: Output 76

The expression `(a[:2] + b[1:])[ -3]` first slices two lists, then joins those slices, then reads the third item from the end.

```
Expression setup
>>> a = [33, 76, 89]
>>> b = [21, "kiran", "harsh"]
>>> (a[:2] + b[1:])[ -3]
76
```

Step	Expression	Result
1	<code>a[:2]</code> takes from the start up to index 2.	<code>[33, 76]</code>
2	<code>b[1:]</code> takes from index 1 to the end.	<code>["kiran", "harsh"]</code>
3	<code>a[:2] + b[1:]</code> joins both sliced lists.	<code>[33, 76, "kiran", "harsh"]</code>
4	<code>[-3]</code> means third item from the end.	<code>76</code>

Negative indexing

In `[33, 76, "kiran", "harsh"]`, -1 is "harsh", -2 is "kiran", and -3 is 76. So the final output is 76.

Day 09/55 takeaway

Lists store many values, can be changed in place, and support indexing, slicing, methods, and numeric helper functions.